p0323r4
std::expected

Published Proposal, 26 November 2017

This version:
http://wg21.link/P0323r4

Issue Tracking:
Inline In Spec

Authors:
Vicente Botet (Nokia)
JF Bastien (Apple)

Audience:
LWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0323r4.bs

Abstract

Utility class to represent expected object: wording and open questions.

Table of Contents

This paper revises [P0323r3] by applying feedback obtained from LEWG and EWG. The previous paper contains motivation, design rationale, implementability information, sample usage, history, alternative designs, and related types. This update only contains wording and open questions because its purpose is twofold:

- Present appropriate wording for including in the Library Fundamentals TS v3.
- List open questions which the TS should aim to answer.

1. Wording

Below, substitute the `�` character with a number or name the editor finds appropriate for the sub-section.

1.1. `�.�` Unexpected objects [*unexpected*]

1.2. `�.�.1` General [*unexpected.general*]

This subclause describes class template `unexpected` that contain objects representing an unexpected outcome.

1.3. `�.�.2` Header `<experimental/unexpected>` synopsis [*unexpected.synop*]

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {
    // 𝔷.𝔷.3, Unexpected object type
    template <class E>
      class unexpected;

    // 𝔷.𝔷.4, Unexpected relational operators
    template <class E>
        constexpr bool
        operator==(const unexpected<E>&, const unexpected<E>&);
    template <class E>
        constexpr bool
        operator!=(const unexpected<E>&, const unexpected<E>&);

}}}
```

A program that needs the instantiation of template `unexpected` for a reference type or `void` is ill-formed.

1.4. `�.�.3` Unexpected object type [*unexpected.object*]

```
template <class E>
class unexpected {
public:
    unexpected() = delete;
    constexpr explicit unexpected(const E&);
    constexpr explicit unexpected(E&&);
    constexpr const E& value() const &;
    constexpr E& value() &;
    constexpr E&& value() &&;
    constexpr E const&& value() const&&;
private:
    E val; // exposition only
};
```

If `E` is void the program is ill formed.

```
constexpr explicit unexpected(const E&);
```

Effects: Build an `unexpected` by copying the parameter to the internal storage `val`.

```
constexpr explicit unexpected(E &&);
```

Effects: Build an `unexpected` by moving the parameter to the internal storage `val`.

```
    constexpr const E& value() const &;
    constexpr E& value() &;
```

Returns: `val`.

```
    constexpr E&& value() &&;
    constexpr E const&& value() const&&;
```

Returns: `move(val)`.

§ 1.5. 20.20.4 Unexpected relational operators [*unexpected.relational_op*]

```
    template <class E>
        constexpr bool operator==(const unexpected<E>& x, const unexpected<E>& y);
```

Requires: `E` shall meet the requirements of *EqualityComparable*.

Returns: `x.value() == y.value()`.

Remarks: Specializations of this function template, for which `x.value() == y.value()` is a core constant expression, shall be constexpr functions.

```
    template <class E>
        constexpr bool operator!=(const unexpected<E>& x, const unexpected<E>& y);
```

Requires: `E` shall meet the requirements of *EqualityComparable*.

Returns: `x.value() != y.value()`.

Remarks: Specializations of this function template, for which `x.value() != y.value()` is a core constant expression, shall be constexpr functions.

§ 1.6. 20.20 Expected objects [expected]

§ 1.7. 𝔼.𝔼.1 In general [*expected.general*]

This subclause describes class template expected that represents expected objects. An `expected<T, E>` object is an object that contains the storage for another object and manages the lifetime of this contained object `T`, alternatively it could contain the storage for another unexpected object `E`. The contained object may not be initialized after the expected object has been initialized, and may not be destroyed before the expected object has been destroyed. The initialization state of the contained object is tracked by the expected object.

§ 1.8. 𝔼.𝔼.2 Header `<experimental/expected>` synopsis [*expected.synop*]

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {
    // 𝔼.𝔼.4, Expected for object types
    template <class T, class E>
        class expected;

    // 𝔼.𝔼.5, Expected specialization for void
    template <class E>
        class expected<void,E>;

    // 𝔼.𝔼.6, unexpect tag
    struct unexpect_t {
       unexpect_t() = default;
    };
    inline constexpr unexpect_t unexpect{};

    // 𝔼.𝔼.7, class bad_expected_access
    template <class E>
       class bad_expected_access;

    // 𝔼.𝔼.8, Specialization for void.
    template <>
       class bad_expected_access<void>;

    // 𝔼.𝔼.9, Expected relational operators
    template <class T, class E>
        constexpr bool operator==(const expected<T, E>&, const expected<T, E>&);
    template <class T, class E>
        constexpr bool operator!=(const expected<T, E>&, const expected<T, E>&);

    // 𝔼.𝔼.10, Comparison with T
    template <class T, class E>
      constexpr bool operator==(const expected<T, E>&, const T&);
    template <class T, class E>
      constexpr bool operator==(const T&, const expected<T, E>&);
    template <class T, class E>
      constexpr bool operator!=(const expected<T, E>&, const T&);
    template <class T, class E>
      constexpr bool operator!=(const T&, const expected<T, E>&);

    // 𝔼.𝔼.10, Comparison with unexpected<E>
    template <class T, class E>
      constexpr bool operator==(const expected<T, E>&, const unexpected<E>&);
    template <class T, class E>
      constexpr bool operator==(const unexpected<E>&, const expected<T, E>&);
    template <class T, class E>
      constexpr bool operator!=(const expected<T, E>&, const unexpected<E>&);
    template <class T, class E>
      constexpr bool operator!=(const unexpected<E>&, const expected<T, E>&);

    // 𝔼.𝔼.11, Specialized algorithms
    void swap(expected<T, E>&, expected<T, E>&) noexcept(see below);

}}}
```

A program that necessitates the instantiation of template `expected<T, E>` with `T` for a reference type or for possibly cv-qualified types `in_place_t`, `unexpect_t` or `unexpected<E>` or `E` for a reference type or `void` is ill-formed.

§ 1.9. 𝔼.𝔼.3 Definitions [*expected.defs*]

An instance of `expected<T, E>` is said to be valued if it contains a value of type `T`. An instance of `expected<T, E>` is said to be unexpected if it contains an object of type `E`.

§ 1.10. 𝔉.𝔉.4 expected for object types [*expected.object*]

```
template <class T, class E>
class expected
{
public:
    typedef T value_type;
    typedef E error_type;
    typedef unexpected<E> unexpected_type;

    template <class U>
        struct rebind {
        using type = expected<U, error_type>;
      };

    // 𝔉.𝔉.4.1, constructors
    constexpr expected();
    constexpr expected(const expected&);
    constexpr expected(expected&&) noexcept(see below);
    template <class U, class G>
        EXPLICIT constexpr expected(const expected<U, G>&);
    template <class U, class G>
        EXPLICIT constexpr expected(expected<U, G>&&);

    template <class U = T>
        EXPLICIT constexpr expected(U&& v);

    template <class... Args>
        constexpr explicit expected(in_place_t, Args&&...);
    template <class U, class... Args>
        constexpr explicit expected(in_place_t, initializer_list<U>, Args&&...);
    template <class G = E>
        constexpr expected(unexpected<G> const&);
    template <class G = E>
        constexpr expected(unexpected<G> &&);
    template <class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template <class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);

    // 𝔉.𝔉.4.2, destructor
    ~expected();

    // 𝔉.𝔉.4.3, assignment
    expected& operator=(const expected&);
    expected& operator=(expected&&) noexcept(see below);
    template <class U = T> expected& operator=(U&&);
    template <class G = E>
        expected& operator=(const unexpected<G>&);
    template <class G = E>
        expected& operator=(unexpected<G>&&) noexcept(see below);

    template <class... Args>
        void emplace(Args&&...);
    template <class U, class... Args>
        void emplace(initializer_list<U>, Args&&...);

    // 𝔉.𝔉.4.4, swap
    void swap(expected&) noexcept(see below);

    // 𝔉.𝔉.4.5, observers
    constexpr const T* operator ->() const;
    constexpr T* operator ->();
    constexpr const T& operator *() const&;
    constexpr T& operator *() &;
    constexpr const T&& operator *() const &&;
    constexpr T&& operator *() &&;
```

```
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr const T& value() const&;
    constexpr T& value() &;
    constexpr const T&& value() const &&;
    constexpr T&& value() &&;
    constexpr const E& error() const&;
    constexpr E& error() &;
    constexpr const E&& error() const &&;
    constexpr E&& error() &&;
    template <class U>
        constexpr T value_or(U&&) const&;
    template <class U>
        T value_or(U&&) &&;

private:
    bool has_val; // exposition only
    union
    {
        value_type val; // exposition only
        unexpected_type unexpect; // exposition only
    };
};
```

Valued instances of `expected<T, E>` where `T` and `E` are of object type shall contain a value of type `T` or a value of type `E` within its own storage. These values are referred to as the contained or the unexpected value of the `expected` object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained or unexpected value. The contained or unexpected value shall be allocated in a region of the `expected<T, E>` storage suitably aligned for the type `T` and `unexpected<E>`. Members `has_val`, `val` and `unexpect` are provided for exposition only. Implementations need not provide those members. `has_val` indicates whether the expected object's contained value has been initialized (and not yet destroyed); when `has_val` is true `val` points to the contained value, and when it is false `unexpect` points to the erroneous value.

`T` must be `void` or shall be object type and shall satisfy the requirements of `Destructible` (Table 27).

`E` shall be object type and shall satisfy the requirements of `Destructible` (Table 27).

§ 1.11. 𝔵.𝔵.4.1 Constructors [*expected.object.ctor*]

```
constexpr expected();
```

Effects: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `T{}` (if `T` is not `void`).

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the default constructor of `T` (nothing if `T` is `void`).

Remarks: If value-initialization of `T` is a constexpr constructor or `T` is `void` this constructor shall be constexpr. This constructor shall be defined as deleted unless `is_default_constructible_v<T>` or `T` is `void`.

```
constexpr expected(const expected& rhs);
```

Effects: If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `*rhs` (if `T` is not `void`).

If `rhs` does not contain a value initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(rhs.error())`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` if `T` is not `void` or by the selected constructor of `unexpected<E>`.

Remarks: This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` or `T` is `void` and `is_copy_constructible_v<E>`. If `is_trivially_copy_constructible_v<T>` is `true` or `T` is `void` and `is_trivially_copy_constructible_v<E>` is `true`, this constructor shall be a constexpr constructor.

```
constexpr expected(expected && rhs) noexcept(see below);
```

Effects: If `rhs` contains a value initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)` (if `T` is not `void`).

If `rhs` does not contain a value initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `std::move(unexpected(rhs.error()))`.

`bool(rhs)` is unchanged.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` if `T` is not `void` or by the selected constructor of `unexpected<E>`.

Remarks: The expression inside `noexcept` is equivalent to: `is_nothrow_move_constructible_v<T> or`T `is ``void` and `is_nothrow_move_constructible_v<E>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T> and is_move_constructible_v<E>`. If `is_trivially_move_constructible_v<T>` is `true` or `T` is `void` and `is_trivially_move_constructible_v<E>` is `true`, this constructor shall be a constexpr constructor.

```
    template <class U, class G>
    EXPLICIT constexpr expected(const expected<U,G>& rhs);
```

Effects: If `rhs` contains a value initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `*rhs` (if `T` is not `void`).

If `rhs` does not contain a value initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(rhs.error())`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` if `T` is not `void` or by the selected constructor of `unexpected<E>`.

Remarks: This constructor shall not participate in overload resolution unless:

- `is_constructible_v<T, const U&>` is `true` or `T` and `U` are `void`,
- `is_constructible_v<E, const G&>` is `true`,
- `is_constructible_v<T, expected<U,G>&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, expected<U,G>&&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, const expected<U,G>&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, const expected<U,G>&&>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<expected<U,G>&, T>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<expected<U,G>&&, T>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<const expected<U,G>&, T>` is `false` or `T` and `U` are `void`, and
- `is_convertible_v<const expected<U,G>&&, T>` is `false` or `T` and `U` are `void`.

The constructor is explicit if and only if `T` is not `void` and `is_convertible_v<U const&, T>` is false or `is_convertible_v<G const&, E>` is false.

```
template <class U, class G>
EXPLICIT constexpr expected(expected<U,G>&& rhs);
```

Effects: If `rhs` contains a value initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)` or nothing if `T`is `void`.

If `rhs` does not contain a value initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `std::move(unexpected(rhs.error()))`. `bool(rhs)` is unchanged

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` if `T` is not `void` or by the selected constructor of `unexpected<E>`.

Remarks: This constructor shall not participate in overload resolution unless:

- `is_constructible_v<T, U&&>` is `true`,
- `is_constructible_v<E, G&&>` is `true`,
- `is_constructible_v<T, expected<U,G>&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, expected<U,G>&&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, const expected<U,G>&>` is `false` or `T` and `U` are `void`,
- `is_constructible_v<T, const expected<U,G>&&>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<expected<U,G>&, T>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<expected<U,G>&&, T>` is `false` or `T` and `U` are `void`,
- `is_convertible_v<const expected<U,G>&, T>` is `false` or `T` and `U` are `void`, and
- `is_convertible_v<const expected<U,G>&&, T>` is `false` or `T` and `U` are `void`.

The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false or `is_convertible_v<G&&, E>` is false.

```
template <class U = T>
EXPLICIT constexpr expected(U&& v);
```

Effects: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::forward<U>(v)`.

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If `T`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `T` is not `void` and `is_constructible_v<T, U&&>` is `true`, `is_same_v<decay_t<U>, in_place_t>` is `false`, `is_same_v<expected<T, E>, decay_t<U>>` is false, and `is_same_v<unexpected<E>, decay_t<U>>` is false. The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false.

```
template <class... Args>
  constexpr explicit expected(in_place_t, Args&&... args);
```

Effects: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the arguments `std::forward<Args>(args)...` if `T` is not `void`.

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the selected constructor of `T` if `T` is not `void`.

Remarks: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `T` is `void`

and `sizeof...Args)==0` or `T` is not `void` and `is_constructible_v<T, Args&&...>`.

```
template <class U, class... Args>
  constexpr explicit expected(in_place_t, initializer_list<U> il, Args&&... args);
```

Effects: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the arguments `il, std::forward<Args>(args)...`.

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `T` is not `void` and `is_constructible<T, initializer_list_v<U>&, Args&&...>`.

```
template <class G = E>
EXPLICIT constexpr expected(unexpected<G> const& e);
```

Effects: Initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `e`.

Postconditions: `*this` does not contain a value.

Throws: Any exception thrown by the selected constructor of `unexpected<E>`

Remark: If `unexpected<E>`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, const G&>`. The constructor is explicit if and only if `is_convertible_v<const G&, E>` is false.

```
template <class G = E>
EXPLICIT constexpr expected(unexpected<G>&& e);
```

Effects: Initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `std::move(e)`.

Postconditions: `*this` does not contain a value.

Throws: Any exception thrown by the selected constructor of `unexpected<E>`

Remark: If `unexpected<E>`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. The expression inside `noexcept` is equivalent to: `is_nothrow_constructible_v<E, G&&>`. This constructor shall not participate in overload resolution unless `is_constructible_v<E, G&&>`. The constructor is explicit if and only if `is_convertible_v<G&&, E>` is false.

```
template <class... Args>
  constexpr explicit expected(unexpect_t, Args&&... args);
```

Effects: Initializes the unexpected value as if direct-non-list-initializing an object of type`unexpected<E>`with the arguments `std::forward<Args>(args)...`.

Postconditions: `*this` does not contain a value.

Throws: Any exception thrown by the selected constructor of `unexpected<E>`

Remarks: If `unexpected<E>`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, Args&&...>`.

```
template <class U, class... Args>
  constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

Effects: Initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with the arguments `il, std::forward<Args>(args)...`.

Postconditions: `*this` does not contain a value.

Throws: Any exception thrown by the selected constructor of `unexpected<E>`.

Remarks: If `unexpected<E>`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, initializer_list<U>&, Args&&...>`.

§ 1.12. 4.2 Destructor [*expected.object.dtor*]

```
~expected();
```

Effects: If `T` is not `void` and `is_trivially_destructible_v<T> != true` and `*this` contains a value, calls `val.~T()`. If `is_trivially_destructible_v<E> != true` and `*this` does not contain a value, calls `unexpect.~unexpected<E>()`.

Remarks: If `T` is `void` or `is_trivially_destructible_v<T> and is_trivially_destructible_v<E>` is `true` then this destructor shall be a trivial destructor.

§ 1.13. 4.3 Assignment [*expected.object.assign*]

```
expected<T, E>& operator=(const expected<T, E>& rhs) noexcept(see below);
```

Effects:

If `*this` contains a value and `rhs` contains a value,

- assigns `*rhs` to the contained value `val` if `T` is not `void`;

otherwise, if `*this` does not contain a value and `rhs` does not contain a value,

- assigns `unexpected(rhs.error())` to the unexpected value `unexpect`;

otherwise, if `*this` contains a value and `rhs` does not contain a value,

- if `T` is `void`
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`. Either
  - The didn't throw, so mark the expected as holding a `unexpected<E>` (which can't throw), or
  - the constructor did throw, an nothing was changed.
- otherwise `T` is not `void`, if `is_nothrow_copy_constructible_v<E>`
  - destroys the contained value by calling `val.T::~T()`,
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`.
- otherwise if `is_nothrow_move_constructible_v<E>`
  - constructs a new `unexpected<E> tmp` on the stack from `*rhs` (this can throw),
  - destroys the contained value by calling `val.T::~T()`,

- initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`.

otherwise as `is_nothrow_move_constructible_v<T>`

- constructs a new `T tmp` on the stack from `*this` (this can throw),
- destroys the contained value by calling `val.T::~T()`,
- initializes the contained value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`. Either,
  - the last constructor didn't throw, so mark the expected as holding a `unexpected<E>` (which can't throw), or
  - the last constructor did throw, so move-construct the `T` from the stack `tmp` back into the expected storage (which can't throw as `is_nothrow_move_constructible_v<T>` is `true`), and rethrow the exception.

otherwise `*this` does not contain a value and `rhs` contains a value

- if `T` is `void` destroys the unexpected value by calling `unexpect.~unexpected<E>()`
- otherwise `T` is not `void`, if `is_nothrow_copy_constructible_v<T>`
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `*rhs`;
- otherwise `T` is not `void`, if `is_nothrow_copy_constructible_v<T>`
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `move(*rhs)`;
- otherwise if `is_nothrow_move_constructible_v<T>`
  - constructs a new `T tmp` on the stack from `*rhs` (this can throw),
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `move(tmp)`;
- otherwise as `is_nothrow_move_constructible_v<E>`
  - constructs a new `unexpected<E> tmp` on the stack from `unexpected(this->error())` (which can throw),
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `*rhs`. Either,
  - the constructor didn't throw, so mark the expected as holding a `T` (which can't throw), or
  - the constructor did throw, so move-construct the `unexpected<E>` from the stack `tmp` back into the expected storage (which can't throw as `is_nothrow_move_constructible_v<E>` is `true`), and rethrow the exception.

Returns: `*this`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: any exception throw by the selected operations.

Remarks: If any exception is thrown, the values of `bool(*this) and bool(rhs)` remain unchanged.

If an exception is thrown during the call to `T`'s or `unexpected<E>`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s or `unexpected<E>`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s or `unexpected<E>`'s copy assignment.

This operator shall be defined as deleted unless

- `T` is `void` and `is_copy_assignable_v<E>` and `is_copy_constructible_v<E>` or
- `T` is not `void` and `is_copy_assignable_v<T>` and `is_copy_constructible_v<T>` and `is_copy_assignable_v<E>` and `is_copy_constructible_v<E>` and (`is_nothrow_move_constructible_v<E>` or `is_nothrow_move_constructible_v<T>`).

```
expected<T, E>& operator=(expected<T, E>&& rhs) noexcept(see below);
```

Effects:

If `*this` contains a value and `rhs` contains a value,

- move assign `*rhs` to the contained value `val` if `T` is not `void`;

otherwise, if `*this` does not contain a value and `rhs` does not contain a value,

- move assign `unexpected(rhs.error())` to the unexpected value `unexpect`;

otherwise, if `*this` contains a value and `rhs` does not contain a value,

- if `T` is `void`
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(move(rhs).error())`. Either
  - the constructor didn't throw, so mark the expected as holding a `unexpected<E>` (which can't throw), or
  - the constructor did throw, and nothing was changed.
- otherwise if `is_nothrow_move_constructible_v<E>`
  - destroys the contained value by calling `val.T::~T()`,
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(move(rhs).error()))`;
- otherwise as `is_nothrow_move_constructible_v<T>`
  - move constructs a new `T tmp` on the stack from `*this` (which can't throw as `T` is nothrow-move-constructible),
  - destroys the contained value by calling `val.T::~T()`,
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected_type<E>` with `unexpected(move(rhs).error()))`. Either,
  - The constructor didn't throw, so mark the expected as holding a `unexpected_type<E>` (which can't throw), or
  - The constructor did throw, so move-construct the `T` from the stack `tmp` back into the expected storage (which can't throw as `T` is nothrow-move-constructible), and rethrow the exception.

otherwise `*this` does not contain a value and `rhs` contains a value,

- if `T` is `void` destroys the unexpected value by calling `unexpect.~unexpected<E>()`
- otherwise, if `is_nothrow_move_constructible_v<T>`
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `*move(rhs)`;
- otherwise as `is_nothrow_move_constructible_v<E>`
  - move constructs a new `unpepected_type<E> tmp` on the stack from `unexpected(this->error())` (which can't throw as `E` is nothrow-move-constructible),
  - destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
  - initializes the contained value as if direct-non-list-initializing an object of type `T` with `*move(rhs)`. Either,

- The constructor didn't throw, so mark the expected as holding a `T` (which can't throw), or
- The constructor did throw, so move-construct the `unexpected<E>` from the stack `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and rethrow the exception.

Returns: `*this`.

Postconditions: `bool(rhs) == bool(*this)`.

Remarks: The expression inside noexcept is equivalent to: `is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>`.

If any exception is thrown, the values of `bool(*this) and bool(rhs)` remain unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. If an exception is thrown during the call to `E`'s copy assignment, the state of its contained unexpected value is as defined by the exception safety guarantee of `E`'s copy assignment.

This operator shall be defined as deleted unless `is_move_constructible_v<T>` and `is_move_assignable_v<T>` and `is_nothrow_move_constructible_v<E>` and `is_nothrow_move_assignable_v<E>`.

```
template <class U>
  expected<T, E>& operator=(U&& v);
```

Effects:

If `*this` contains a value, assigns `forward<U>(v)` to the contained value `val`;

otherwise, if `is_nothrow_constructible_v<T, U&&>`

- destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
- initializes the contained value as if direct-non-list-initializing an object of type `T` with `forward<U>(v)` and
- set `has_val`to `true`;

otherwise as `is_nothrow_constructible_v<E>`

- move constructs a new `unexpected<E> tmp` on the stack from `unexpected(this->error())` (which can't throw as `E` is nothrow-move-constructible),
- destroys the contained value by calling `unexpect.~unexpected<E>()`,
- initializes the contained value as if direct-non-list-initializing an object of type `T` with `forward<U>(v)`. Either,
  - the constructor didn't throw, so mark the expected as holding a `T` (which can't throw), that is set `has_val` to `true`, or
  - the constructor did throw, so move construct the `unexpected<E>` from the stack `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.

Returns: `*this`.

Postconditions: `*this` contains a value.

Remarks: If any exception is thrown, the value of `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment.

This function shall not participate in overload resolution unless:

- `is_same_v<T,void>` is `false` and

- `is_same_v<expected<T,E>, decay_t<U>>` is `false` and
- `conjunction_v<is_scalar<T>, is_same<T, decay_t<U>>>` is `false`,
- `is_constructible_v<T, U>` is `true`,
- `is_assignable_v<T&, U>` is `true` and
- `is_nothrow_move_constructible_v<E>` is `true`.

```
expected<T, E>& operator=(unexpected<E> const& e) noexcept(see below);
```

Effects:

If `*this` does not contain a value, assigns `unexpected(rhs.error())` to the unexpected value `unexpect`;

otherwise,

- destroys the contained value by calling `val.~T()` if `T` is not `void`,
- initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(forward<expected<T, E>>(rhs))` and set `has_val` to `false`.

Returns: `*this`.

Postconditions: `*this` does not contain a value.

Remarks: If any exception is thrown, value of valued remains unchanged.

This signature shall not participate in overload resolution unless `is_nothrow_copy_constructible_v<E>` and `is_assignable_v<E&, E>`.

```
expected<T, E>& operator=(unexpected<E> && e);
```

Effects:

If `*this` does not contain a value, move assign `unexpected(rhs.error())` to the unexpected value `unexpect`;

otherwise,

- destroys the contained value by calling `val.~T()` if `T` is not `void`,
- initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(move(forward<expected<T, E>>(rhs)))` and set `has_val` to `false`.

Returns: `*this`.

Postconditions: `*this` does not contain a value.

Remarks: If any exception is thrown, value of valued remains unchanged.

This signature shall not participate in overload resolution unless `is_nothrow_move_constructible_v<E>` and `is_move_assignable_v<E&, E>`.

```
void expected<void,E>::emplace();
```

Effects:

If `*this` doesn't contains a value

- destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
- set `has_val`to `true`

Postconditions: `*this` contains a value.

Throws: nothing

```
template <class... Args>
  void emplace(Args&&... args);
```

Effects:

If `*this` contains a value, assigns the contained value `val` as if constructing an object of type `T` with the arguments `std::forward<Args>(args)...`

otherwise, if `is_nothrow_constructible_v<T, Args&&...>`

- destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
- initializes the contained value as if direct-non-list-initializing an object of type `T` with `std::forward<Args>(args)...` and
- set `has_val` to `true`;

otherwise if `is_nothrow_move_constructible_v<T>`

- constructs a new `T tmp` on the stack from `std::forward<Args>(args)...` (which can throw),
- destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
- initializes the contained value as if direct-non-list-initializing an object of type `T` with `move(tmp)` (which can not throw) and
- set `has_val` to `true`;

otherwise as `is_nothrow_move_constructible_v<E>`

- move constructs a new `unexpected<E> tmp` on the stack from `unexpected(this->error())` (which can't throw),
- destroys the unexpected value by calling `unexpect.~unexpected<E>()`,
- initializes the contained value as if direct-non-list-initializing an object of type `T` with `std::forward<Args>(args)...`. Either,
  - the constructor didn't throw, so mark the expected as holding a `T` (which can't throw), that is set `has_val` to `true`, or
  - the constructor did throw, so move-construct the `unexpected<E>` from the stack `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the selected assignment of `T`.

Remarks: If an exception is thrown during the call to `T`'s assignment, nothing changes.

This signature shall not participate in overload resolution unless `is_no_throw_constructible_v<T, Args&&...>`.

```
template <class U, class... Args>
  void emplace(initializer_list<U> il, Args&&... args);
```

Effects: if `*this` contains a value, assigns the contained value `val` as if constructing an object of type `T` with the arguments `il, std::forward<Args>(args)...`, otherwise destroys the unexpected value by calling `unexpect.~unexpected<E>()` and initializes the contained value as if constructing an object of type `T` with the arguments `il, std::forward<Args>(args)...`.

Postconditions: `*this` contains a value.

Throws: Any exception thrown by the selected assignment of `T`.

Remarks: If an exception is thrown during the call to `T`'s assignment nothing changes.

The function shall not participate in overload resolution unless: `T` is not `void` and `is_no_throw_constructible_v<T, initializer_list<U>&, Args&&...>`.

§ 1.14. 𝔼.𝔼.4.4 Swap [*expected.object.swap*]

```
void swap(expected<T, E>& rhs) noexcept(see below);
```

> ISSUE 1 Check `swap` Effects.

Effects: if `*this` contains a value and `rhs` contains a value,

- calls `using std::swap; swap(val, rhs.val)`,

otherwise if `*this` does not contain a value and `rhs` does not contain a value,

- calls `using std::swap; swap(err, rhs.err)`,

otherwise if `*this` does not contains a value and `rhs` contains a value,

- calls `rhs.swap(*this)`,

otherwise, `*this` contains a value and `rhs` does not contains a value,

- if `T` is `void`
  - initializes the unexpected value as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(move(rhs))`. Either
  - the constructor didn't throw, so mark the expected as holding a `unexpected<E>`, destroys the unexpected value by calling `rhs.unexpect.~unexpected<E>()` set `rhs.has_val` to `true`.
  - the constructor did throw, rethrow the exception.
- otherwise `T` is not `void`, if `is_nothrow_move_constructible_v<E>`,
  - the unexpected value of `rhs` is moved to a temporary variable `tmp` of type `unpected_type`,
  - followed by destruction of the unexpected value as if by `rhs.unexpect.unexpected<E>::~unexpected<E>()`,
  - the contained value of `rhs` is direct-initialized from `std::move(*other)`,
  - followed by destruction of the contained value as if by `rhs.val->T::~T()`,
  - the unexpected value of `this` is direct-initialized from `std::move(tmp)`, after this, `this` does not contain a value; and `rhs` contains a value.
- otherwise if `is_nothrow_move_constructible_v<T>`,
  - the contained value of `rhs` is moved to a temporary variable `tmp` of type `T`,
  - followed by destruction of the contained value as if by `rhs.val.T::~T()`,
  - the unexpected value of `rhs` is direct-initialized from `unexpected(std::move(other.error()))`,
  - followed by destruction of the unexpected value as if by `rhs.unexpect->unexpected<E>::~unexpected<E>()`,
  - the contained value of `this` is direct-initialized from `std::move(tmp)`, after this, `this` does not contain a value; and `rhs` contains a value.

Throws: Any exceptions that the expressions in the Effects clause throw.

> ISSUE 2 Adapt `swap` Remarks once Effects are good.

Remarks: The expression inside noexcept is equivalent to: `is_nothrow_move_constructible_v<T> and noexcept(swap(declval<T&>(), declval<T&>())) and is_nothrow_move_constructible_v<E> and noexcept(swap(declval<E&>(), declval<E&>()))`. The function shall not participate in overload resolution unless: LValues of type `T` shall be `Swappable`, LValues of type `E` shall be `Swappable` and

`is_move_constructible_v<E>` `is_move_constructible_v<E>` `is_move_constructible_v<E>` or `is_move_constructible_v<T>`.

1.15. ﷺ.4.5 Observers [*expected.object.observe*]

```
constexpr const T* operator->() const;
T* operator->();
```

Requires: `*this` contains a value.

Returns: `&val`.

Remarks: Unless `T` is a user-defined type with overloaded unary `operator&`, the first operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr const T& operator *() const&;
T& operator *() &;
```

Requires: `*this` contains a value.

Returns: `val`.

Remarks: The first operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr T&& operator *() &&;
constexpr const T&& operator *() const&&;
```

Requires: `*this` contains a value.

Returns: `move(val)`.

Remarks: This operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr explicit operator bool() noexcept;
```

Returns: `has_val`.

Remarks: This operator shall be a constexpr function.

```
    constexpr bool has_value() const noexcept;
```

Returns: `has_val`.

Remarks: This function shall be a constexpr function.

```
constexpr void expected<void, E>::value() const;
```

Throws: `bad_expected_access(err)` if `*this` does not contain a value.

```
constexpr const T& expected::value() const&;
constexpr T& expected::value() &;
```

Returns: `val`, if `*this` contains a value.

Throws: `bad_expected_access(err)` if `*this` does not contain a value.

Remarks: These functions shall be constexpr functions. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr T&& expected::value() &&;
constexpr const T&& expected::value() const&&;
```

Returns: `move(val)`, if `*this` contains a value.

Throws: `bad_expected_access(err)` if `*this` does not contain a value.

Remarks: These functions shall be constexpr functions. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr const E& error() const&;
constexpr E& error() &;
```

Requires: `*this` does not contain a value.

Returns: `unexpect.value()`.

Remarks: The first function shall be a constexpr function.

```
constexpr E&& error() &&;
constexpr const E&& error() const &&;
```

Requires: `*this` does not contain a value.

Returns: `move(unexpect.value())`.

Remarks: The first function shall be a constexpr function.

```
template <class U>
  constexpr T value_or(U&& v) const&;
```

Effects: Equivalent to `return bool(*this) ? **this : static_cast<T>(std::forward<U>(v));`.

Remarks: If `is_copy_constructible_v<T>` and `is_convertible_v<U&&, T>` is false the program is ill-formed.

```
template <class U>
  T value_or(U&& v) &&;
```

Effects: Equivalent to `return bool(*this) ? std::move(**this) : static_cast<T>(std::forward<U>(v));`.

Remarks: If `is_move_constructible_v<T>` and `is_convertible_v<U&&, T>` is false the program is ill-formed.

§ 1.16. 𝔵.𝔵.6 `unexpect` tag [*expected.unexpect*]

```
struct unexpect_t {
    explicit unexpect_t() = default;
};
inline constexpr unexpect_t unexpect{};
```

§ 1.17. 𝔵.𝔵.7 Template Class `bad_expected_access` [*expected.bad_expected_access*]

```
template <class E>
class bad_expected_access : public bad_expected_access<void> {
public:
    explicit bad_expected_access(E);
    virtual const char* what() const noexcept override;
    E& error() &;
    const E& error() const&;
    E&& error() &&;
    const E&&  error() const&&;
private:
    E val; // exposition only
};
```

ISSUE 3 Wondering if we just need an `const &` overload as we do for `system_error`.

The template class `bad_expected_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of a unexpected expected object.

```
bad_expected_access::bad_expected_access(E e);
```

Effects: Constructs an object of class `bad_expected_access` storing the parameter.

Postconditions: `what()` returns an implementation-defined NTBS.

```
    const E& error() const&;
    E& error() &;
```

Effects: Equivalent to: `return val;`

```
    E&& error() &&;
    const E&& error() const &&;
```

Effects: Equivalent to: `return move(val);`

```
virtual const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.

§ 1.18. ﷿.﷿.7 Template Class `bad_expected_access<void>` [*expected.bad_expected_access_base*]

```
template <>
class bad_expected_access<void> : public exception {
public:
    explicit bad_expected_access();
};
```

The template class `bad_expected_access<void>` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of a unexpected expected object.

§ 1.19. ﷿.﷿.8 Expected Relational operators [*expected.relational_op*]

```
    template <class T, class E>
        constexpr bool operator==(const expected<T, E>& x, const expected<T, E>& y);
```

Requires: `T` (if not `void`) and `unexpected<E>` shall meet the requirements of *EqualityComparable*.

Returns: If `bool(x) != bool(y)`, `false`; otherwise if `bool(x) == false`, `unexpected(x.error()) == unexpected(y.error())`; otherwise `true` if `T` is `void` or `*x == *y` otherwise.

Remarks: Specializations of this function template, for which `T` is `void` or `*x == *y` and `unexpected(x.error()) == unexpected(y.error())` are core constant expression, shall be constexpr functions.

```
    template <class T, class E>
        constexpr bool operator!=(const expected<T,    E>& x, const expected<T, E>& y);
```

Requires: `T` (if not `void`) and `unexpected<E>` shall meet the requirements of *EqualityComparable*.

Returns: If `bool(x) != bool(y)`, `true`; otherwise if `bool(x) == false`, `unexpected(x.error()) != unexpected(y.error())`; otherwise `true` if `T` is `void` or `*x != *y`.

Remarks: Specializations of this function template, for which `T` is `void` or `*x != *y` and `unexpected(x.error()) != unexpected(y.error())` are core constant expression, shall be constexpr functions.

§ 1.20. 🔗.🔗.9 Comparison with `T` [*expected.comparison_T*]

```
    template <class T, class E> constexpr bool operator==(const expected<T, E>& x, const T&
v);
    template <class T, class E> constexpr bool operator==(const T& v, const expected<T, E>&
x);
```

Requires: `T` is not `void` and the expression `*x == v` shall be well-formed and its result shall be convertible to `bool`. [Note: `T` need not be *EqualityComparable*. - end note]

Effects: Equivalent to: `return bool(x) ? *x == v : false;`.

```
    template <class T, class E> constexpr bool operator!=(const expected<T, E>& x, const T&
v);
    template <class T, class E> constexpr bool operator!=(const T& v, const expected<T, E>&
x);
```

Requires: `T` is not `void` and the expression `*x == v` shall be well-formed and its result shall be convertible to `bool`. [Note: `T` need not be *EqualityComparable*. - end note]

Effects: Equivalent to: `return bool(x) ? *x != v : false;`.

§ 1.21. 🔗.🔗.10 Comparison with `unexpected<E>` [*expected.comparison_unexpected_E*]

```
    template <class T, class E> constexpr bool operator==(const expected<T, E>& x, const un
expected<E>& e);
    template <class T, class E> constexpr bool operator==(const unexpected<E>& e, const exp
ected<T, E>& x);
```

Requires: The expression `unexpected(x.error()) == e` shall be well-formed and its result shall be convertible to `bool`. [Note: `E` need not be *EqualityComparable*. - end note]

Effects: Equivalent to: `return bool(x) ? true : unexpected(x.error()) == e;`.

```
    template <class T, class E> constexpr bool operator!=(const expected<T, E>& x, const un
expected<E>& e);
    template <class T, class E> constexpr bool operator!=(const unexpected<E>& e, const exp
ected<T, E>& x);
```

Requires: The expression `unexpected(x.error()) != e` shall be well-formed and its result shall be convertible to `bool`. [Note: `E` need not be *EqualityComparable*. - end note]

Effects: Equivalent to: `return bool(x) ? false : unexpected(x.error()) != e;`.

1.22. 🔒.🔒.11 Specialized algorithms [*expected.specalg*]

```
template <class T, class E>
void swap(expected<T, E>& x, expected<T, E>& y) noexcept(noexcept(x.swap(y)));
```

Effects: Calls `x.swap(y)`.

Remarks: This function shall not participate in overload resolution unless `T` is `void` or `is_move_constructible_v<T>` is `true`, `is_swappable_v<T>` is `true` and `is_move_constructible_v<E>` is `true` and `is_swappable_v<E>` is `true`

2. Open Questions

`std::expected` is a vocabulary type with an opinionated design and a proven record under varied forms in a multitude of codebases. Its current form has undergone multiple revisions and received substantial feedback, falling roughly in the following categories:

1. **Ergonomics**: is this *the right way* to expose such functionality?
2. **Disappointment**: should we expose this in the Standard, given C++'s existing error handling mechanisms?
3. **STL usage**: should the Standard Template Library adopt this class, at which pace, and where?

LEWG and EWG have nonetheless reached consensus that a class of this general approach is probably desirable, and the only way to truly answer these questions is to try it out in a TS and ask for explicit feedback from developers. The authors hope that developers will provide new information which they'll be able to communicate to the Committee.

Here are open questions, and questions which the Committee thinks are settled and which new information can justify revisiting.

2.1. Ergonomics

1. Name:
    - Is `expected` the right name?
    - Does it express intent both as a consumer and a producer?
2. Is `E` a salient property of `expected`?
3. Is `expected<void, E>` clear on what it expresses as a return type?
4. Would it make sense for `expected` to support containing *both* `T` and `E` (in some designs, either one of them being optional), or is this usecase better handled by a separate proposal?
5. Is the order of parameters `<T, E>` appropriate?
6. Is usage of `expected` "viral" in a codebase, or can it be adopted incrementally?
7. Comparisons:
    - Are `==` and `!=` useful?
    - Should other comparisons be provided?
    - What usages of `expected` mandate putting instances in a `map`, or other such container?
    - Should `hash` be provided?

- What usages of `expected` mandate putting instances in an `unordered_map`, or other such container?
- Should `expected<T, E>` always be comparable if `T` is comparable, even if `E` is not comparable?

8. Error type `E`:
   - `E` has no default. Should it?
   - Should `expected` be specialized for particular `E` types such as `exception_ptr`, and how?
   - Should `expected` handle `E` types with a built-in "success" value any differently, and how?
   - `expected` is not implicitly constructible from an `E`, even when unambiguous from `T`, because as a vocabulary type it wants unexpected error construction to be verbose, and require hopping through an `unexpected`. Is the verbosity extraneous?
9. Does usage of this class cause a meaningful performance impact compared to using error codes?
10. The accessor design offers a terse unchecked dereference operator (expected to be used alongside the implicit `bool` conversion), as well as `value()` and `error()` accessors which are checked. Is that a gotcha, or is it similar enough to classes such as `optional` to be unsurprising?
11. Is `bad_expected_access` the right thing to throw?
12. Should some members be `[[nodiscard]]`?

§ 2.2. Disappointment

C++ already supports exceptions and error codes, `expected` would be a third kind of error handling.

1. where does `expected` work better than either exceptions or error handling?
2. `expected` was designed to be particularly well suited to APIs which require their immediate caller to consider an error scenario. Do it succeed in that purpose?
3. Do codebases successfully compose these three types of error handling?
4. Is debuggability any harder?
5. Is it easy to teach C++ as a whole with a third type of error handling?

§ 2.3. STL Usage

1. Should `expected` be used in the STL at the same time as it gets standardized?
2. Where, considering `std2` may be a good place to change APIs?

§ References

§ Informative References

[P0323r3]
Vicente J. Botet Escriba. Utility class to represent expected object. URL: https://wg21.link/p0323r3

§ Issues Index

ISSUE 1 Check `swap` Effects. ↵

ISSUE 2 Adapt `swap` Remarks once Effects are good. ↵

ISSUE 3 Wondering if we just need an `const &` overload as we do for `system_error`. ↵